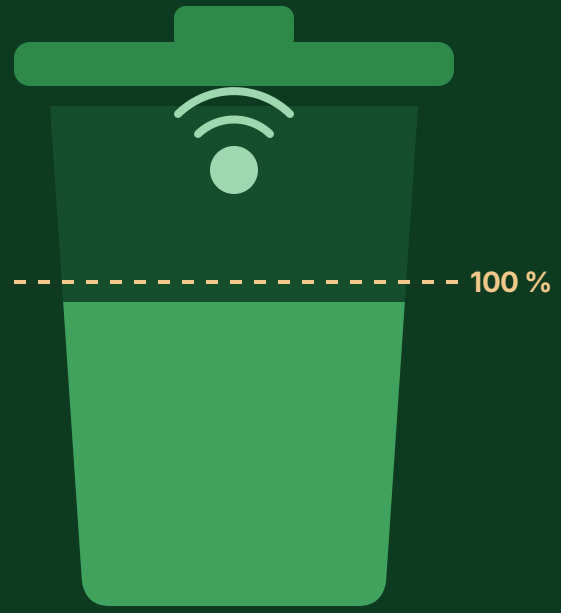
PRÉSENTATION TECHNIQUE · ÉQUIPE BAC CONNECT

Le backend de Bac Connect

Comment le serveur reçoit les mesures des bacs, calcule leur remplissage et décide quand il faut les ramasser.



Elda Wamba · backend Node.js / Express / MySQL

29 septembre 2026 · Collège Ahuntsic

01 · VUE D'ENSEMBLE

Le backend est le cerveau du système

Il reçoit les mesures brutes des capteurs, les transforme en **kilogrammes** et en **pourcentage de remplissage**, décide si un bac doit être ramassé, et sert tout ça à l'application mobile.

📡 ESP32 dans le bac

Envoie toutes les **3 s** la lecture brute de la balance (HX711) et la distance du capteur (VL53L0X). Il ne reçoit **jamais** d'ordre en retour.

🧠 Backend (ce projet)

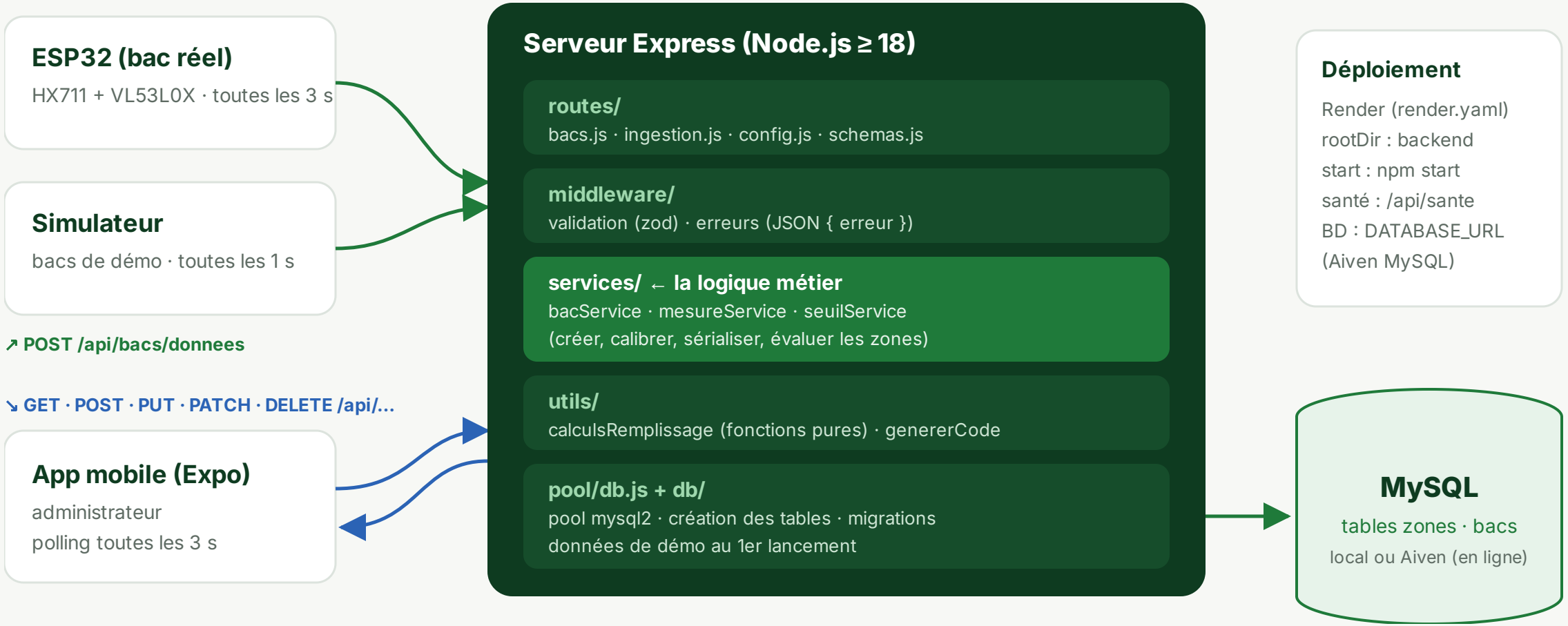
Valide, convertit, stocke dans MySQL et applique la **logique de ramassage**. Tous les calculs sont faits ici, jamais dans l'app.

📱 App administrateur

Lit l'état des bacs toutes les **3 s** (polling), crée des bacs et des zones, lance le calibrage, met un bac en attente.

> Règle d'or : **le frontend n'invente aucune valeur**. Il affiche `estPlein`, `poidsPlein`, `alertePoidsExcessif` tels que le backend les calcule.

Qui parle à qui



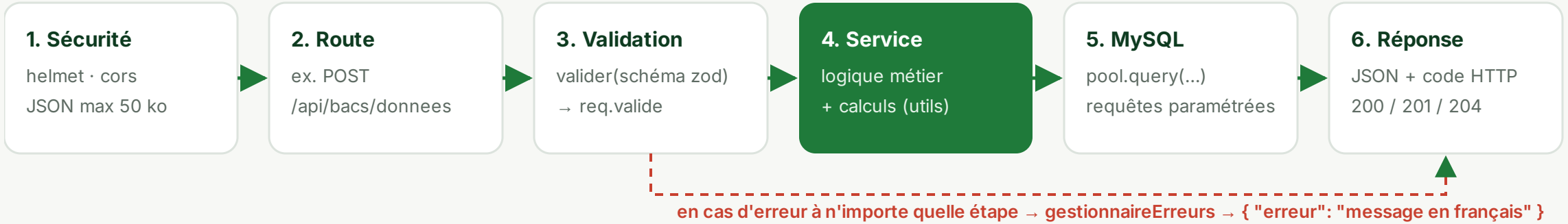
Stack et rôle de chaque dossier

Paquet	Pourquoi
`express` 4	serveur HTTP / API REST
`mysql2`	connexion MySQL (pool)
`zod`	validation des données reçues
`helmet` · `cors`	en-têtes de sécurité, accès depuis l'app
`express-rate-limit`	anti-inondation des capteurs
`dotenv`	configuration via `.env`
`nodemon` · `supertest`	dev auto-reload · tests API

```
backend/
├── server.js        # démarre : BD → écoute port 3000 → simulateur
├── app.js           # Express : sécurité, routes, erreurs
├── routes/          # les URL de l'API + schémas zod
├── middleware/      # valider() · gestionnaireErreurs
├── services/        # logique métier (bacs, mesures, zones)
├── utils/           # calculs purs, génération des codes BAC-XXXXXX
├── pool/db.js       # UNE connexion MySQL partagée
├── db/              # tables, migrations, données de démo
├── simulator/       # faux capteurs pour la démo
├── tests/           # 16 tests (npm test)
└── .env             # mots de passe — jamais sur GitHub
```

Séparation **app.js / server.js** : les tests chargent l'application sans ouvrir de port ni lancer le simulateur.

Chaque requête traverse les mêmes étapes




```
// routes/bacs.js
routeur.post('/:id/calibrage',
  valider(schemas.idBac, 'params'),
  valider(schemas.calibrage),
  asynchrone(async (req, res) => {
    res.status(200).json(
      await bacService.enregistrerCalibrage(
        req.valide.params.id, req.valide.body));
  }));
```

- `valider()` refuse la requête (400) **avant** qu'elle touche la base.
- `asynchrone()` renvoie toute erreur async vers le gestionnaire central : pas de `try/catch` dans chaque route.
- Les **routes restent minces** : toute la logique est dans `services/`.
- Les noms de champs JSON sont **exactement** ceux du frontend et du sketch ESP.

Deux tables : zones et bacs

zones	
`id`, `nom`	ex. Montréal, Laval
`seuil_pourcentage`	% de bacs pleins qui rend la zone **critique** (défaut 80)
`statut_signal`	`normal` ou `critique`
`seuil_plein_pourcentage`	conservé, n'est plus utilisé pour décider

bacs	
`id`, `code` (unique)	`BAC-5MTDUY` = identifiant programmé dans l'ESP32
`nom`, `type_dechet`, `zone_id`	déchets / recyclage / compost · lien vers la zone
`statut`	`normal` ou `en_attente` (vidage en cours)
`poids_kg`, `profondeur_cm`, `derniere_lecture_brute`, `dernier_contact`	**dernière mesure** (pas d'historique)
`zero_brut`, `facteur_calibration`, `profondeur_vide`, `profondeur_pleine`, `poids_max_kg`	**calibrage** du bac
`est_simule`	1 = bac de démo alimenté par le simulateur

Au démarrage, `initialiserBaseDeDonnees()` crée les tables si besoin, ajoute les colonnes manquantes (migrations) et insère les **données de démo** une seule fois.

Un bac est **calibré** seulement si `profondeur_vide`, `profondeur_pleine` et `poids_max_kg` sont remplis. Sinon : aucun pourcentage, jamais « plein ».

Ce qui se passe quand l'ESP32 envoie une mesure

1. **POST** `/api/bacs/donnees` avec `{"code":"BAC-5MTDUY","lectureBrute":123456,"profondeurCm":42.5}`
2. **Anti-inondation** : max 120 mesures / minute par code, sinon **429**.
3. **Validation zod** : profondeur entre 0 et 1000 cm, nombres valides.
4. Le bac est retrouvé par son **code** (inconnu → 404).
5. Bac `en_attente` ? La mesure est **ignorée** : `{"ignore":true}`.
6. Lecture brute **convertie en kg** grâce au calibrage, puis `UPDATE bacs` (poids, profondeur, dernier contact).
7. La **zone** du bac est réévaluée (normal / critique).

```
// services/mesureService.js (simplifié)
async function enregistrerMesure(code, brute, profCm) {
  const bac = await obtenirBacParCode(code);
  if (!bac) throw new ErreurHttp(404, 'Code de bac inconnu');
  if (bac.statut === 'en_attente') return { ignore: true };

  const poidsKg = calculerPoidsReel(
    brute, bac.zero_brut, bac.facteur_calibration);

  await pool.query(`UPDATE bacs SET poids_kg = ?,
    derniere_lecture_brute = ?, profondeur_cm = ?,
    dernier_contact = ? WHERE id = ?`, [...]);

  if (bac.zone_id) await evaluerZone(bac.zone_id);
  return { ignore: false };
}
```

> On ne garde que la **dernière** mesure. Si aucun contact depuis **5 min** (300 s), l'API marque `donneePerimee: true` .

Des fonctions pures dans utils/calculsRemplissage.js

Aucun accès à la base, aucun effet de bord : faciles à tester et à relire.

Poids réel (kg)

```
poids = (lectureBrute - zeroBrut) ÷ facteurCalibration
```

Arrondi au gramme, jamais négatif (une petite dérive de la balance n'affiche pas −0,2 kg).

Remplissage par profondeur

```
% = (profVide - profMesurée) ÷ (profVide - profPleine)
```

Borné entre 0 et 1. Le capteur est au couvercle : plus il y a de déchets, plus la distance est petite.

Ratio de poids

```
ratio = poidsKg ÷ poidsMaxKg      // NON plafonné
```

Peut dépasser 1 : indispensable pour détecter 120 %. Une version plafonnée (`calculerPourcentagePoids`) sert seulement aux jauges.

Pourcentage affiché

```
pourcentageRemplissage = max(%profondeur, %poids)
```

L'API renvoie aussi les deux valeurs séparées pour la visualisation du bac (niveau + tube de poids).

Exemple : profondeur vide 100 cm, pleine 10 cm, mesure 55 cm → (100 − 55) ÷ 90 = **50 %**.

Quand faut-il ramasser un bac ?

Situation	estPlein	poidsPlein	alerte	Dans l'app
Profondeur ≥ **100 %**	✅	—	—	ROUGE Prêt pour la collecte
Poids **100** → **120 %**	❌	✅	❌	ORANGE Poids plein – profondeur normale
Poids ≥ **120 %**	✅	❌	✅	ROUGE Ramassage nécessaire – poids excessif
Sinon	❌	❌	❌	VERT

> Seuils **fixes dans le code**, identiques pour toutes les zones. Un bac à 97 % n'est **pas** encore prêt.

```
const SEUIL_PROFONDEUR_RAMASSAGE = 1;    // 100 %
const SEUIL_POIDS_PLEIN          = 1;    // 100 %
const SEUIL_POIDS_ALERTE         = 1.2;  // 120 %

function evaluerRamassage(pctProfondeur, ratioPoids) {
  const poidsExcessif =
    ratioPoids >= SEUIL_POIDS_ALERTE;
  const poidsPlein =
    ratioPoids >= SEUIL_POIDS_PLEIN && !poidsExcessif;
  const necessiteRamassage =
    pctProfondeur >= SEUIL_PROFONDEUR_RAMASSAGE
    || poidsExcessif;
  return { necessiteRamassage, poidsPlein,
           alertePoidsExcessif: poidsExcessif };
}
```

Dans `bacService`, `estPlein = necessiteRamassage`, forcé à `false` si le bac est en attente ou pas calibré. Le nom `estPlein` est gardé pour ne rien casser (stats, filtre « Prêts pour la collecte »).

Une zone devient critique quand trop de bacs sont pleins

```
% pleins = bacs pleins ÷ bacs actifs calibrés

si % pleins ≥ seuil_pourcentage de la zone
   → statut_signal = 'critique'
sinon
   → statut_signal = 'normal'
```

Exemple · zone Montréal, seuil 80 %

5 bacs actifs, 4 pleins → **80 %** ≥ 80 % → CRITIQUE

5 bacs actifs, 3 pleins → **60 %** < 80 % → NORMAL

Quand `evaluerZone()` est-elle relancée ?

- à **chaque mesure** reçue d'un bac de la zone
- quand on **modifie les seuils** de la zone
- quand un bac est **mis en attente** ou remis en service
- après un **calibrage**
- quand un bac **change de zone** (les **deux** zones sont recalculées)
- quand un bac est **supprimé**

> Les bacs **en attente** et **non calibrés** ne comptent pas. La même fonction `evaluerRamassage` décide si un bac est plein : bac et zone ne peuvent pas se contredire.

5 étapes pour transformer des chiffres bruts en kg et en %

1. **Tarage à vide** : lecture brute, bac vide → `zeroBrut`
2. **Poids étalon** : lecture brute avec un objet de poids connu → `lectureEtalon` + `poidsEtalonKg`
3. **Profondeur à vide** : distance du capteur, bac vide → `profondeurVide`
4. **Distance pleine** (saisie) → `profondeurPleine`
5. **Poids maximal** (saisie) → `poidsMaxKg`

Les étapes 1 à 3 peuvent aussi être saisies à la main dans l'app si l'ESP32 n'envoie rien.

```
// Formule standard HX711 — jamais saisie à la main
facteur_calibration =
  (lectureEtalon - zeroBrut) ÷ poidsEtalonKg

// ex. (11000 - 0) ÷ 11 kg = 1000 unités par kg
```

Refusé par le backend (400)

- une des 6 valeurs manquante
- profondeur à vide ≤ 0 (capteur non branché)
- lecture étalon = tarage (poids non détecté)
- distance pleine ≥ profondeur à vide

`POST /api/bacs/:id/calibrage`

11 · RÉFÉRENCE DE L'API

Les endpoints

Méthode	URL	Rôle	Utilisé par
GET	`/api/sante`	Serveur et base de données OK ?	Render, dev
POST	`/api/bacs/donnees`	Recevoir une mesure (code, lectureBrute, profondeurCm)	ESP32, simulateur
GET	`/api/bacs?zoneId=`	Liste des bacs (bac réel en tête), filtre par zone optionnel	App
GET	`/api/bacs/resume`	Totaux : actifs, en attente, pleins, par type de déchet	Tableau de bord
GET	`/api/bacs/:id`	Détail d'un bac (lecture live pendant le calibrage)	App
POST	`/api/bacs`	Créer un bac → le code `BAC-XXXXXX` est généré	App
PUT	`/api/bacs/:id`	Modifier nom, type, zone, position	App
DELETE	`/api/bacs/:id`	Supprimer un bac	App
PATCH	`/api/bacs/:id/statut`	`normal` ↔ `en_attente`	App
POST	`/api/bacs/:id/calibrage`	Enregistrer le calibrage (6 valeurs)	App
GET POST	`/api/zones`	Lister / créer des zones	App
PUT DELETE	`/api/zones/:id`	Modifier le seuil / supprimer (409 si la zone contient des bacs)	App

Des erreurs toujours lisibles par l'app

Toute erreur répond en JSON `{ "erreur": "message en français" }`, le format que lit l'app. Jamais de page HTML (qui ferait planter `response.json()`).

Code	Quand
400	données invalides (zod), JSON cassé, zone introuvable
404	route, bac ou code de bac inconnu
409	supprimer une zone qui contient des bacs
429	plus de 120 mesures / min pour un même bac
500	erreur interne → message générique, détail dans la console

Protections en place

- `helmet` : en-têtes HTTP de sécurité
- corps JSON limité à **50 ko**
- **rate limit** par code de bac sur l'ingestion
- requêtes SQL **paramétrées** (`?`) : pas d'injection SQL
- `.env` dans le `.gitignore` : mots de passe jamais sur GitHub
- codes de bac sans `0/O/1/I` pour éviter les confusions

> Limite connue : **pas d'authentification sur l'API** (un seul administrateur, connexion gérée par l'app). À prévoir avant une vraie mise en production.

Un seul vrai bac, les autres sont simulés

🟢 Le bac réel

`BAC-5MTDUY` · « Bac Prototype – Cégep Ahuntsic » · compost · zone Montréal.

Code déjà programmé dans l'ESP32 : **ne jamais le régénérer**. Toujours affiché en tête de liste.

🤖 Les bacs simulés

Marqués `est_simule = 1`, déjà calibrés. Le simulateur leur envoie une mesure **chaque seconde**, par la même URL que l'ESP32.

Ils se remplissent puis sont « ramassés » au hasard : un cycle réaliste pour la démo.

🗺️ Zones par défaut

Montréal (80 %), Laval (75 %), Centre-du-Québec (85 %).

Insérées au 1er lancement seulement : jamais de doublons au redémarrage.

Le simulateur démarre **automatiquement** avec le serveur. Pour le couper : `SIMULATION_AUTO=false` dans `.env`. Pour le lancer seul : `npm run simuler`.

Un bac **créé dans l'app** n'est jamais simulé (`est_simule = 0`) : il attend les vraies mesures d'un ESP32.

Lancer, tester, déployer

1 · En local

```
# MySQL doit tourner
brew services start mysql

cd backend
npm install
cp .env.example .env
# remplir DB_USER / DB_PASSWORD
npm run dev

# vérifier :
http://localhost:3000/api/sante
```

2 · Tests

```
npm test

# base dédiée : bacconnect_test
# (créée et vidée automatiquement)

✔ 16 / 16 tests OK
```

Ils vérifient le contrat avec l'app et l'ESP : noms des champs, format des erreurs, calculs, ramassage, zones, calibrage.

3 · En ligne

```
# Render → New → Blueprint
# (fichier render.yaml)

DATABASE_URL=mysql://…
  # copiée depuis Aiven
CODE_BAC_REEL=BAC-5MTDUY
SIMULATION_AUTO=true
```

Render surveille `/api/sante` pour savoir si le serveur est en vie.

Le serveur écoute sur `0.0.0.0:3000` : le téléphone (Expo Go) et l'ESP32 le joignent par l'IP de l'ordinateur, **sur le même Wi-Fi**.

À RETENIR

5 idées clés

1. Le backend fait **tous** les calculs, l'app ne fait qu'afficher.

2. Une requête = route → validation zod → service → MySQL → JSON.

3. Ramassage : profondeur ≥ 100 % **ou** poids ≥ 120 %. Entre 100 et 120 % de poids : simple avertissement.

4. Chaque mesure, calibrage ou changement de statut **réévalue la zone**.

5. Sans calibrage, pas de pourcentage. Sans contact depuis 5 min, la donnée est **périmée**.

→ Code : `github.com/bacconnect/bacconnect-app` · dossier `backend/`

Questions ?